Mini C Compiler: Six-Part Code Guide

The C source is kept in one file, compiler.c, so it is easy to build. Comments inside that file mark the six parts. Each person's section below follows those same markers and explains the code in simple language.

Part 1 — Data and lexer setup

This part starts at the top of compiler.c and continues through the beginning of lex, before the PART 2 marker.

- The #include lines load standard C tools. stdio.h handles input and output, stdlib.h provides general utilities, string.h handles text, and ctype.h checks letters, digits, and spaces.

- The MAX_... constants set the maximum number of tokens, text length, variables, and intermediate instructions.

- Token stores the text of one token and the source line where it appeared.

- The arrays named tokens, variables, and code keep tokens, declared variable names, and generated instructions. Their counters track how much has been stored.

- add_token puts a token and its source line into the token array. It stops if the array is full.

- is_keyword compares a word with a list of language words. The current parser does not call this helper; it checks token text directly.

- lex starts scanning at the first character and tracks the current source line. It skips spaces and new lines, prepares a word buffer, and skips // comments.

Part 2 — Recognizing tokens

This part starts at the PART 2 marker inside lex and ends when lex adds the <END> token.

- Quoted text is collected and added as one string token. The backslash check allows escaped characters inside the string.

- A name starts with a letter or underscore. Later characters can also be digits. Words such as int and while are read as names; the parser later recognizes their meaning.

- A number is read by collecting consecutive digits.

- Two-character operators such as ++, ==, <=, and && are checked before single-character operators.

- Single-character operators and punctuation include +, =, ;, braces, and parentheses.

- If a character does not match a supported token, the lexer prints an error and continues scanning.

- <END> marks the end of the token list so the parser knows when to stop.

- In the visualizer, tokens are shown as numbered rows with their source line, making it easier to find where each one came from.

Part 3 — Symbol table and expression helpers

This part starts at the PART 3 marker, after the lexer.

- is_number checks whether a token begins with a digit.
- is_declared searches the variable list for a name.
- declare_var reports an error if the name was declared already. Otherwise, it adds the name to the list.

- use_var reports an error if code uses a name that was never declared.
- peek looks at the current token without moving forward. accept checks a token and consumes it when it matches. expect requires a token and reports a syntax error if it is missing.

- emit saves an intermediate instruction.
- new_temp and new_label create names such as t1 for calculations and L1 for jump destinations.

- primary reads a number, variable, or parenthesized expression. It checks that variables were declared. On an invalid value, it reports an error and uses 0 so parsing can continue.

- term reads multiplication, division, and remainder operations. For example, a b becomes an instruction like t1 = a b.

Part 4 — Addition, comparisons, and basic statements

This part starts at the PART 4 marker before additive.

- additive reads terms joined with + or -. It saves each calculation in a temporary variable.

- expression reads additive expressions and comparisons such as <, ==, or >=. Its result can be used as a condition.

- block reads statements between { and }. It stops at the closing brace or at the end of the token list.

- At the beginning of statement, the code recognizes the supported int main() { ... } program wrapper. Other function declarations are not supported.

- The declaration case reads code such as int count = 3;. It adds the variable to the symbol table, then reads and saves an initial value if one is present.

Part 5 — If statements and loops

This part starts at the PART 5 marker inside statement.

- The if case reads a condition and body. It creates an ifFalse jump and labels to represent the if path and optional else path.

- The while case creates a label at the loop start, checks the condition, reads the body, and jumps back to the start. A false condition jumps to the end label.

- The for case reads an initializer and condition. It saves the increment tokens, reads the loop body, then handles a basic increment such as i++, i--, or a simple assignment.

- The for implementation is intentionally small. It does not support every expression allowed in the full C language.

Part 6 — Print, return, assignments, and phase output

This part starts at the PART 6 marker inside statement and includes the rest of the file.

- The printf case supports a quoted string, one simple value, or one %d integer placeholder with a value. It turns these into print instructions.

- The return case reads either return; or return followed by an expression, then emits a return instruction.

- The assignment case reads a variable assignment, ++, or --. It checks that the variable was declared first.

- If a statement is not recognized, the compiler reports a syntax error and moves to the next token so it can continue.

- main reads source text from standard input, runs the lexer, parses the statements, and prints the tokens, syntax/semantic result, symbol table, and intermediate code.

- run_intermediate walks through the generated instructions. It follows labels and jumps, performs integer calculations, and collects text produced by print instructions. A step limit stops accidental infinite loops.

- The program output is shown after the intermediate code. For the sample, it displays Result: 10.

- The compiler runs generated instructions only when there are no errors. It returns 0 for success and 1 when it finds errors. The web server uses this result to show success or failure.

Build and run on Windows

Open PowerShell in the project folder and build the C compiler:

```
gcc .\compiler.c -o .\mini_compiler.exe
```

Start the local web server:

```
python .\server.py
```

Keep that terminal open, then open http://127.0.0.1:8765 in a browser. Click Load sample, then Compile and view phases. The page shows tokens, syntax/semantic checks, the symbol table, and intermediate code separately. It also displays the result of running the generated instructions in a Program output panel.

To recreate the PDF after editing this guide, run python .\make_pdf.py in PowerShell from the project folder.

Tools used

- C and GCC/MinGW: build the compiler executable from compiler.c.
- Python 3: runs the local server that connects the browser and C program.
- HTML, CSS, and JavaScript: create and operate the browser interface.

- A web browser: lets a user enter code and view the compiler phases.